

First Look at Geant4

Greg Willmott

gw00431@surrey.ac.uk

What is Geant4?

The screenshot shows the Geant4 website homepage. The browser address bar displays 'https://geant4.web.cern.ch'. The navigation menu includes 'About', 'Download', 'Documentation', 'User Forum', 'Bug Reports', 'Events', and 'Contact Us'. The main header features the Geant4 logo and the title 'Geant4' with a subtitle: 'Toolkit for the simulation of the passage of particles through matter. Its areas of application include high energy, nuclear and accelerator physics, as well as studies in medical and space science.' A 'Getting started' button is visible. Below the header, there are four columns: 'Get started' (with a link 'I'm ready to start!'), 'Download' (with a link 'open source license' and 'Latest: 11.3.2'), 'Docs' (with a link 'Read documentation'), and 'News' (with dates '25 Apr 2025' and '21 Mar 2025' and links 'Release 11.3.2' and 'Release 11.3.1').

Toolkit

Not a complete code, but a starting point

Monte Carlo methods

Each particle's life is random, but physical

Particles through matter

Sounds quite basic when put this way

Applications

Fields that want to simulate detector response

Key terms

Nomenclature	Explanation
<i>Step</i> , <i>Event</i> , <i>Run</i> , <i>Track</i> * (hierarchical)	<i>One vertex of a particle life/history,</i> <i>The entire particle life/history,</i> <i>The collection of all Events</i> <i>*Similar to Events</i>
Physics List	List of functions which enable models built into base Geant4
Particle gun / General Particle Source (GPS)	The two ways of starting an event i.e. the primary particle
Run macro	List of commands to do at run time
<i>Solid</i> , <i>Logical</i> , <i>Physical</i> volumes (hierarchical)	<i>The type of shape you want,</i> <i>The type and material,</i> <i>The type, material, and location in the world</i>

Links

Github repo of all my simulations:

<https://github.com/GregWillmott/geant4-simulations>

Book for Application Developers

<https://indico.cern.ch/event/647154/contributions/2714212/attachments/1529029/2397032/BookForApplicationDevelopers.pdf>

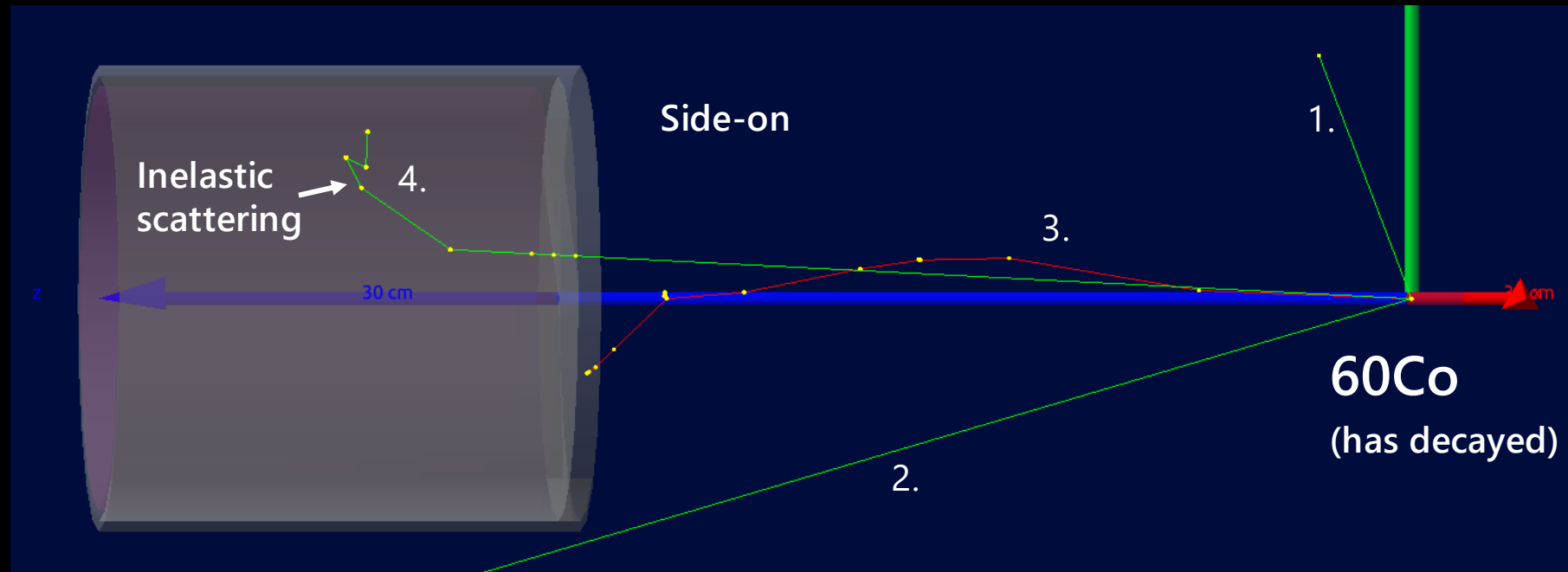
Intrinsic efficiency curve of HPGe's (used in Gammasphere analysis)

<https://iopscience.iop.org/article/10.1088/1742-6596/1055/1/012010/pdf>

Official installation guide

https://geant4.web.cern.ch/documentation/dev/ig_html/InstallationGuide/index.html

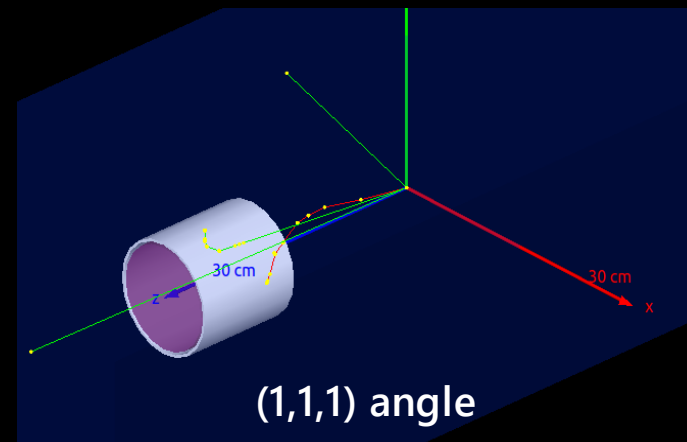
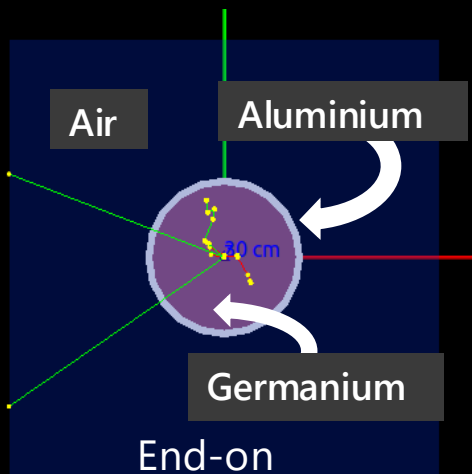
Example event



1. Neutrino or gamma
2. Neutrino or gamma
3. Negatively charged particle (beta-)
4. Gamma ray

- No charge (γ or ν)
- Negative ion (e^-)
- Positive ion (nucleus, e^+)
- Step (interaction point)

Hitting world boundary kills particle



Material definitions in the appendix of the Book for Application Developers

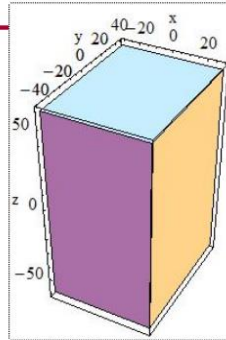
Geometry

CSG (Constructed Solid Geometry)

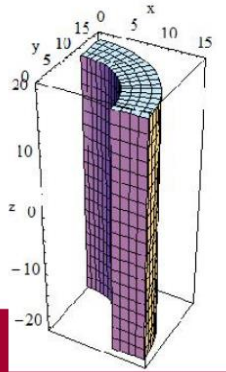
G4Box, G4Sphere, G4Tubs (cylinder), G4Polyhedra...

CSG: G4Box, G4Tubs

```
G4Box(const G4String &pname, // name
      G4double half_x, // X half size
      G4double half_y, // Y half size
      G4double half_z); // Z half size
```



```
G4Tubs(const G4String &pname, // name
      G4double pRmin, // inner radius
      G4double pRmax, // outer radius
      G4double pDz, // Z half length
      G4double pSphi, // starting Phi
      G4double pDphi); // segment angle
```



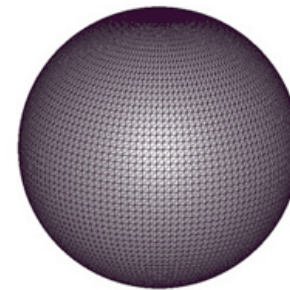
CAD (as in engineering models)

Models can be decomposed into triangles (STL, STEP format)

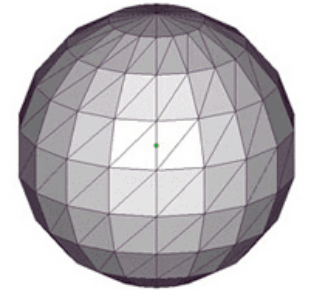
GDML (special type of XML)



Original



Good fidelity



Bad fidelity

http://geant4.in2p3.fr/IMG/pdf_Lecture-Geometry.pdf

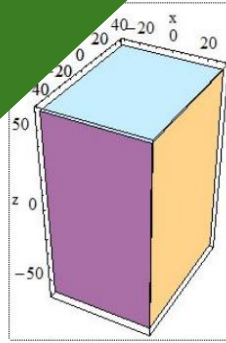
Geometry

CSG (Constructed Solid Geometry)

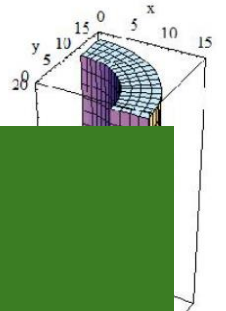
G4Box, G4Sphere, G4Tubs (cylinder), G4Disk

CSG: G4Box, G4Tubs

```
G4Box(const G4String &pname, // name
      G4double half_x, // half size in x
      G4double half_y, // half size in y
      G4double half_z, // half size in z)
```



```
G4Tubs(const G4String &pname, // name
      G4double rmin, // inner radius
      G4double rmax, // outer radius
      G4double zmin, // z position at bottom
```



Easier to learn and implement

Faster runtimes

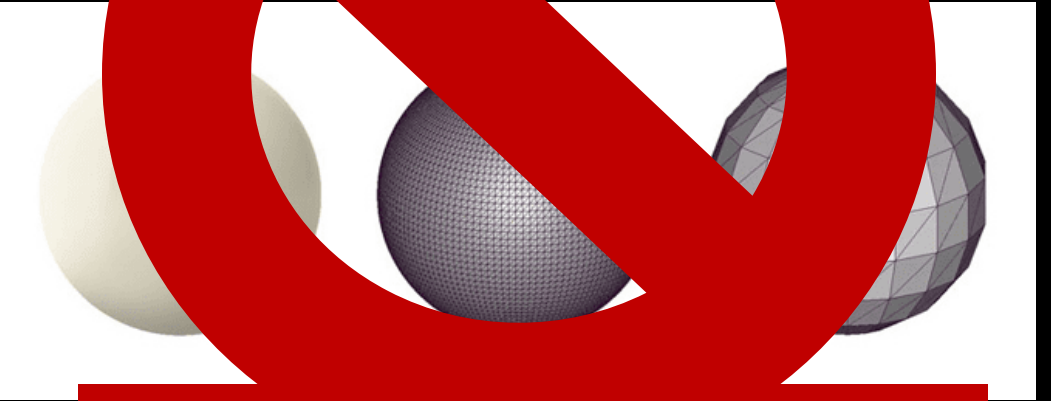
But...

Some setups are too complicated to model in a reasonable amount of time

<http://geant4.cern.ch/WWW/pdflib/Lecture-Geometry.pdf>

CAD (as in engineering models)

Models can be exported to various formats (STL, STEP for CAD, GDML for GEANT4, XML)



Loading takes longer and particle transport takes longer (more surfaces)
Hard to implement (GDML is the most straightforward but the CAD has to be converted to this niche format)

Closed Surface Geometry

Solid volume

Choose the shape

Give the arguments for its size (often in half-lengths e.g. a `G4Box(1*cm, 1*cm, 1*cm)` would be 2x2x2cm)

Some solid volumes act on solid volumes you've already made (`G4SubtractionSolid`, `G4UnionSolid`, `G4IntersectionSolid`)

Logical volume

Takes solid volume as argument along with a `G4Material*` object

Use `G4NistManager*` to call upon premade materials or define your own earlier, down to a `G4Element*` or `G4Isotope*` level of precision

Physical volume

Takes logical volume as argument and a `G4RotationMatrix` and a `G4ThreeVector` for rotation and position about the world's origin

`G4PVPlacement()` command

The world is the first volume. Subsequent volumes have the world as an argument so they become daughters (prevents overlap issues)



CSG is built up in 3 layers

The boolean solid volumes take a vector and a rotation, which is applied to volume 2. Volume 1's centre is the origin.

Primary particle generation

Particle gun

Have to recompile to make changes,
Pick particle type ("gamma", "e-", "ion"...),
Pick kinetic energy and direction,
Can use random numbers instead of fixed values for a more realistic beam

General Particle Source

Does the same thing but the code is in a run time macro,
I label them gps_xxxxxxx.mac,
The main run macro executes this second macro,
Commands available to modify energy, direction, particle type,
Can have multiple sources with a probability to decide which gets picked

```
// Class constructor definition
OurPrimaryGeneratorAction::OurPrimaryGeneratorAction()
{
    // Boolean put in place to go between particle gun and source
    ifGeneralSource = true;

    if(ifGeneralSource == false){
        // Default behaviour: shoot gamma 1173 keV along Z axis.
        G4int n_particle = 1; // 1 per shot
        fParticleGun = new G4ParticleGun(n_particle);

        // Properties
        G4ParticleTable* particleTable = G4ParticleTable::GetParticleTable();
        G4String particleName;
        G4ParticleDefinition* particle
        = particleTable->FindParticle(particleName="gamma");
        fParticleGun->SetParticleDefinition(particle);
        fParticleGun->SetParticleMomentumDirection(G4ThreeVector(0.,0.,1.));
        fParticleGun->SetParticleEnergy(1.173*MeV);
    }
    else{
        // Else: use general particle source
        fGPS = new G4GeneralParticleSource;
    }
}
```

Set the particle type to ion, and give the Z and A of a radioactive isotope to have the moment after a decay as the start of the event

Physics lists

Adding in models

The models are off by default to save on computing resource. There are completed lists that can be turned on with one line of code, but my sims choose the specific packages.

Verbosity: level of detail put into the terminal on what physics has happened during transport. Zero is silent.

Can also modify cuts (min length of a vertex before the particle is deemed to be too low energy and is 'absorbed', otherwise transport would go on forever).

More info:

<https://geant4-userdoc.web.cern.ch/UsersGuides/PhysicsReferenceManual/html/index.html>
<https://indico.cern.ch/event/776050/contributions/3237925/attachments/1789252/2914238/PhysLists.pdf>

```
// Define class constructor. Expected args: verbosity
OurPhysics::OurPhysics(int verbosity) : G4VModularPhysicsList()
{
    SetVerboseLevel(verbosity);

    // -----
    // EM Physics
    // -----
    // Most detailed EM Phys package, available alternatives include:
    // G4EmLivermorePhysics, G4EmPenelopePhysics, G4EmStandardPhysics_option1,
    RegisterPhysics(new G4EmStandardPhysics_option4(verbosity));

    //Synchrotron Radiation & GN Physics
    // G4EmExtraPhysics* emExtraPhysics = new G4EmExtraPhysics(verbosity);
    // Use Livermore low-energy nuclear data for gamma-nuclear interactions
    // emExtraPhysics->LENDGammaNuclear(true);
    // RegisterPhysics(emExtraPhysics);

    // Radioactive Physics
    RegisterPhysics(new G4DecayPhysics(verbosity));
    RegisterPhysics(new G4RadioactiveDecayPhysics(verbosity));

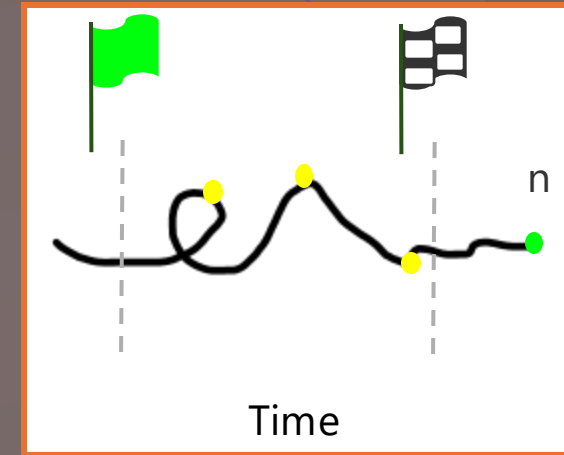
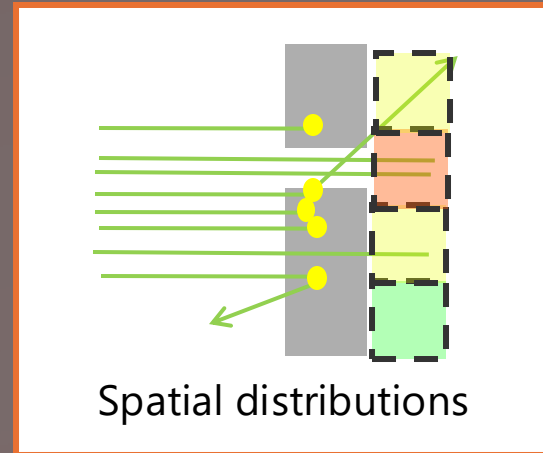
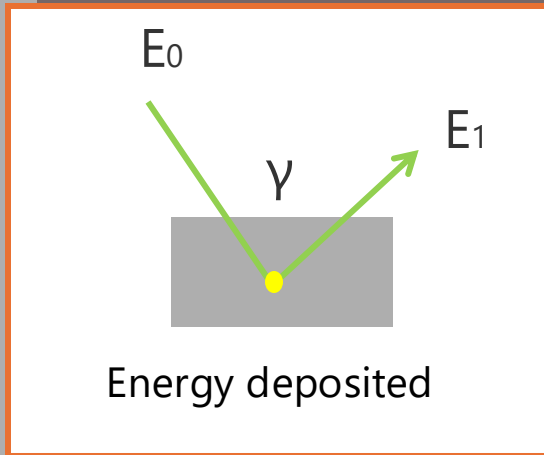
    // Hadron Elastic scattering
    // RegisterPhysics(new G4HadronElasticPhysicsLEND(verbosity));
    RegisterPhysics(new G4HadronElasticPhysicsHP(verbosity));

    // Hadron inelastic scattering
    // RegisterPhysics(new G4HadronPhysicsShieldingLEND());
    RegisterPhysics(new G4HadronPhysicsShielding(verbosity));

    // Stopping Physics
    RegisterPhysics(new G4StoppingPhysics(verbosity));

    // Ion Physics
    RegisterPhysics(new G4IonElasticPhysics(verbosity));
}
```

What can be output?



Spectroscopy /
detector response

Dose maps

Dosimetry

Investigate
kinematics

n-tof method

Counting experiments

lifetimes

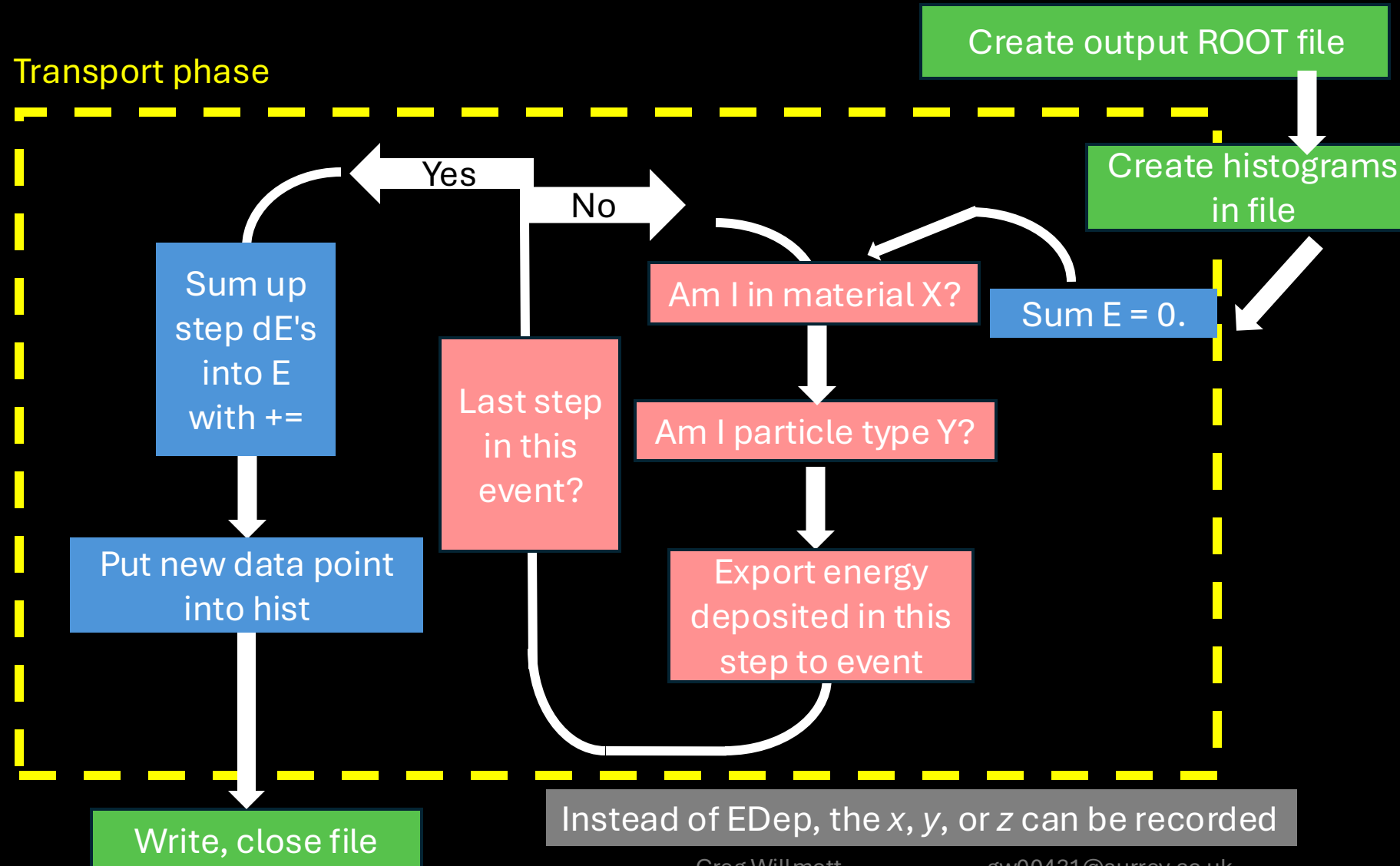
2.7 cm
2.7 cm
2.7 cm

**By default, Geant will transport
your particles but output no data!**

How to get an output?

Typical logic

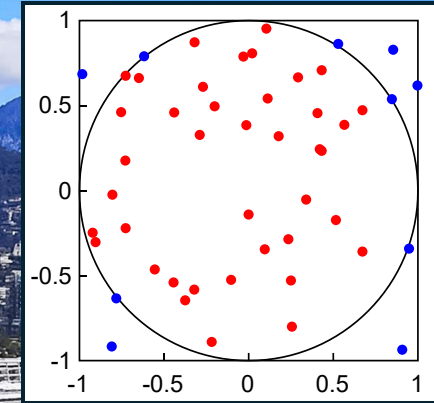
Transport phase



The Monte Carlo aspect

Classic example

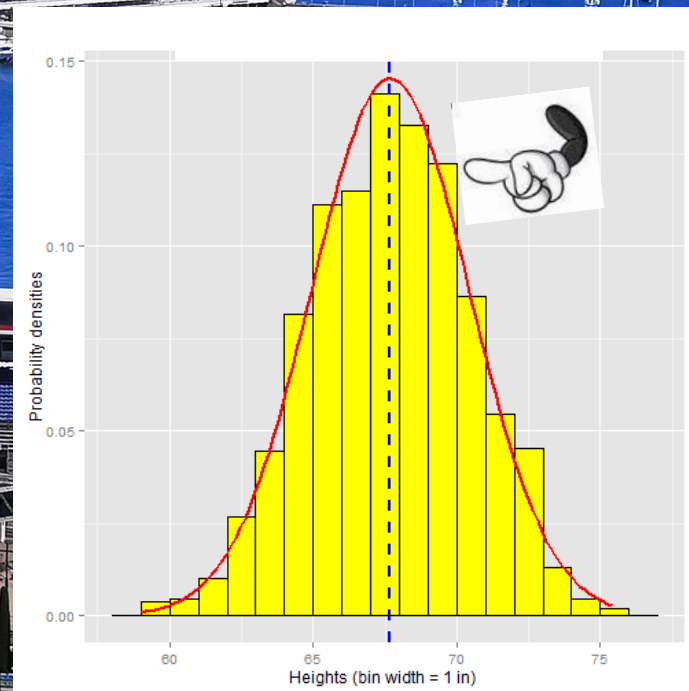
Estimate the value of π from the fraction of random dots in the circle compared to the total



Geant4

The direction the particle travels, the amount of energy loss, the processes which are chosen to happen (i.e. Compton scattering, pair production...) are decided by sampling probability distribution functions that are made from real cross section data and models.

Each particle appears to behave randomly. Go from 10 to 1,000,000 particles and the overall behaviour will start to converge. *Hopefully* the trend is physical.



What can/can't Geant model?

Models include

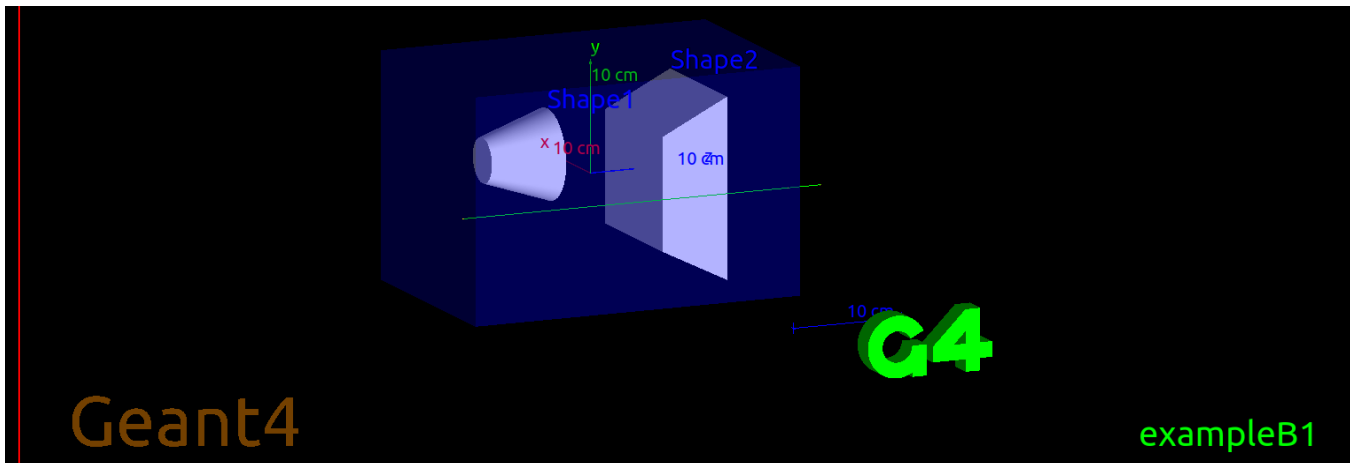
- Solid angle coverage i.e. geometric efficiency
- Compton scattering
- Pair production
- Decay (from excited states)
- Radioactive decay (ions with specific Z, A values)
- Photoelectric effect
- Energy loss
- Elastic scattering
- Atomic relaxation
- Bremsstrahlung
- Cherenkov cones, optical properties
- Motion in E or B fields
- Hadronic, neutrino and solid state physics models
- ...

But these are missing from my sims/base Geant4

- Majority of electron production as the gamma is absorbed (the cut kills the gamma prematurely)
- Efficiency of charge collection in a HPGe detector
- Nuclear physics reactions (d,p), (d,t) etc
- Decay and cross section data is not infinite
- Jack has a specific simulation where the primary particles for the simulation are the calculated products of a (d,p) reaction
- NPTool has these nuclear physics reactions but has its own disadvantages
- Energy loss is not a replacement for SRIM

Installation and setup

- Standard procedure (get source code, make a build and install folder, cmake and modify compilation flags to en/disable features, make, make install)
- Needs ROOT to be installed first
- Autoinstall bash script is included with these slides. It won't work very well if your environment is arranged differently to mine, but it includes all necessary commands and what packages need to be sudo apt-get installed. It also has the URLs of the Github repos.
- Once Geant is built and installed the first test is usually to navigate to Geant/install/share/Geant4/examples/basic/B1, create a build folder for this example sim, compile with cmake and make, and run the executable



- Press the green play button at the top to shoot one particle

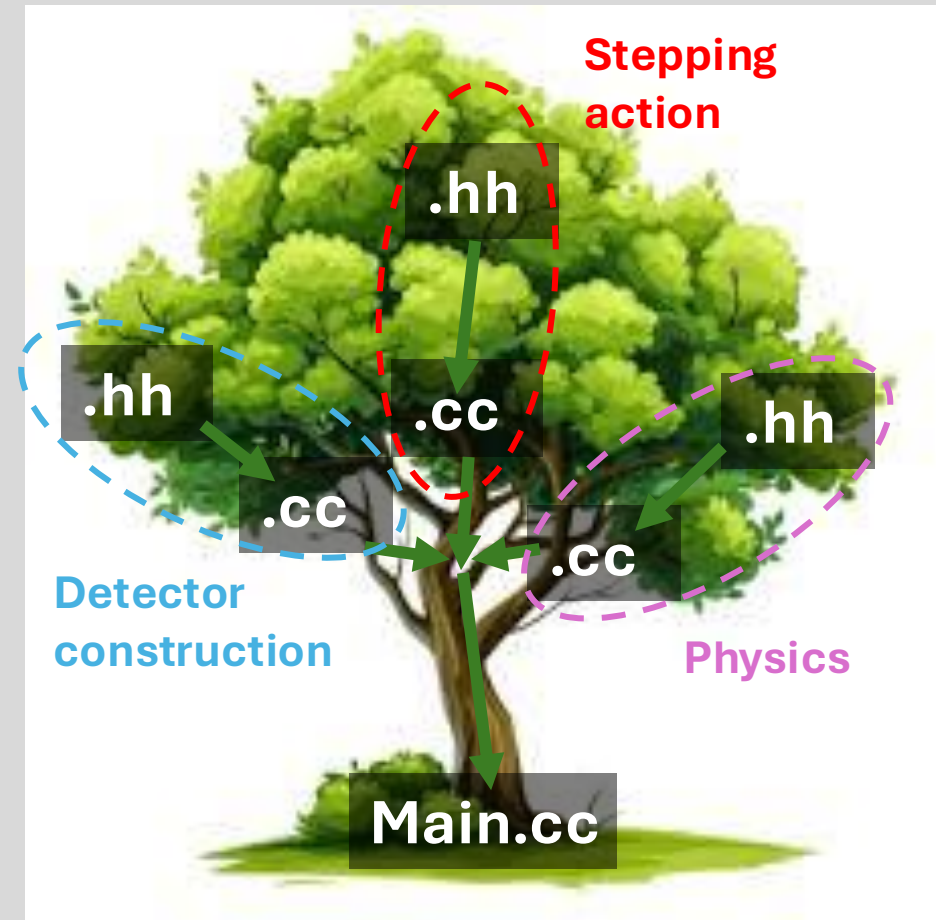
Code layout

Many .cc and .hh files

Apart from the main file which contains the `int main()` function, all source code is kept in .cc files which each have their own corresponding .hh header file. The other files are macros or the CMakeLists file.

Geant4 convention

All example codes stick to defining one class per .cc file. Sometimes there is complication when classes depend on other classes e.g. the action initialisation does nothing but call upon the step, event, and running actions
(instructions for the step, event, and run levels of iteration), and also the primary generator action *(the particle gun)*.



Gamma spectroscopy example

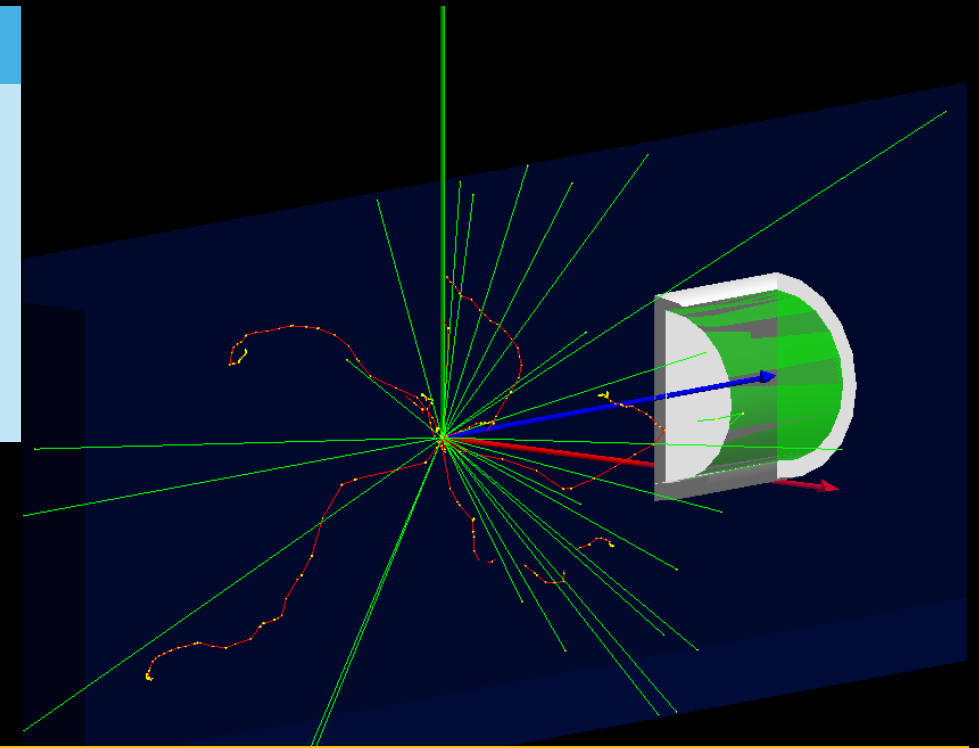
- The included simulation is a germanium crystal inside an aluminium casing. The source is the products of a ^{60}Co beta decay (1173, 1332 keV gammas, a beta and a neutrino)
- The energy deposited in the germanium is the data carried into the ROOT file
- Compilation and execution instructions are in the README.

Exercises once the sim is running

- Can you change the material of one of the volumes to something else from the NIST database? See appendix of the 'book for application developers'
- Can you change the size of the Germanium crystal?
- Can you swap to a different General Particle Source? Or make ^{152}Eu ?

Advanced exercise

- Can you add a second 1D histogram which records the Z coordinate from the last step of each event? To find different functions belonging to the classes google 'G4Step class' to bring up apc.u-paris.fr



Screenshot after 10 events

Example sim output

RunMacroBatch.mac runs 1 mil 60Co particles.

Log Y axis, draw option 'hist'

